

Parent-approved playlists, local playback

YouTube supplies approved material. Jellyfin supplies the child experience.

PREFERRED DESIGN

Adopt the proposed pipeline: parent selects a playlist, approves an exact snapshot, a managed worker retrieves and normalizes permitted videos into a dedicated RAID subfolder, and Jellyfin indexes only completed assets. The kids app keeps its current player, Next, Shuffle and progress model.

Why this is a stronger technical fit

Decision	Consequence
Playback stays local	Once published, an episode no longer depends on YouTube stream URL resolution, a web player or WAN availability. The downloader can fail while existing cached videos keep playing.
HEVC first; AV1 measured	Use an HEVC/H.265 + AAC MP4 profile as the conservative starting point. Preserve a compatible source when practical. Treat AV1 as an optional tested profile, not an assumed improvement. [S20, S21]
Keep familiar navigation	First version: add only the published Shows directory as an extra folder in the existing Kid TV library. Keep the exact Kid TV/Kid Movie binding. A third YouTube library and new top-level UI become optional later. [S1]
Scoped RAID exception	The user now permits writes in a dedicated cache subfolder. All existing media and every other RAID path remain protected. The worker must receive only that narrow write capability.

The parent still owns the playlist boundary

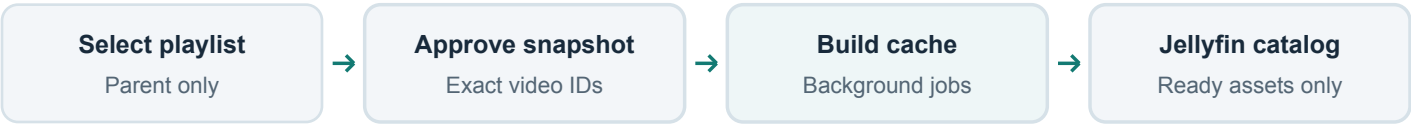
Approving a playlist means approving its current video IDs. Later additions remain pending. The published local file is the approved artifact; subsequent upstream edits do not replace it automatically. A playlist is not a standing instruction to download future content.

Acquisition is a separate permission issue: YouTube restrictions and rightsholder rights remain applicable to downloading and offline copies. This design does not infer authorization from an API key or Premium account. [S11, S17]

Design revision dated 8 October 2026. No cache folder, downloader, transcode job, server library or client change was created. The exact RAID volume, folder, quotas and server encoding capabilities are still unverified.

Select, review, queue, publish

An adult web interface manages imports; children see only ready Jellyfin items.



Admin screen	Controls and resulting behavior
Find + preview	Jellyfin already has web library administration at http://1531server.local:8096/web/ . Add a small parent import page beside it. Paste/select a playlist; search can follow later. Show original source and channel identity, duration, language and preview. [S1]
Review snapshot	Enumerate every playlist page and inspect video metadata. Parent excludes unwanted clips or compilations and approves the exact remaining IDs in bulk. A partial fetch cannot publish. Matching titles and made-for-kids labels do not establish family suitability. [S5, S6]
Queue approved IDs	Record the draft hash, ordered IDs and approval revision. Workers receive individual admitted IDs, never an unrestricted playlist URL for automatic expansion. Jobs report downloading, normalizing, verifying, ready, indexing or failed.
Publish ready assets	Generate files, local metadata and artwork outside the indexed tree. Verify the bundle, move it into the published tree, then refresh Jellyfin. Show the parent which assets became visible; no broken placeholders or half-written episodes.
Maintain the set	New playlist members enter a changes inbox. Upstream edits do not overwrite approved files. Existing cache remains usable during download failures. Removing or replacing a cached item is an explicit parent action.

Frozen approval and stable identity

Store collection UUID, source playlist/channel/video IDs, approval revision, local asset digest, stable relative path and Jellyfin item mapping. Recheck the revision before publication. If approval was revoked while a job ran, keep the result out of the published tree.

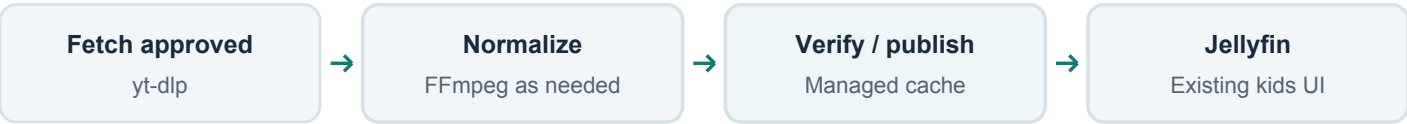
PARENT AUTHORITY

Validate a real administrator session for imports and publication. A child PIN is only a local UI gate. Kids cannot launch jobs, submit URLs or expand playlists. Service credentials stay on the server; no child Google login or cookie access.

A YouTube video can be trimmed without changing its URL. A verified local file avoids silent replacement, but only after an authorized acquisition. Where the Data API is used, refresh/delete its cached metadata under the applicable retention rules; retain parent-authored approval records separately. [S18, S11]

Download once; encode only when useful

The output profile is a playback decision, not simply a newer-codec preference.



Step / profile	Recommended behavior
Fetch	yt-dlp retrieves available audio/video streams. FFmpeg handles merging or encoding; changing a container does not by itself produce HEVC. Pin tested tool versions and track each job by video ID. Current YouTube support also requires its supported JavaScript tooling. [S19]
Default output	Candidate baseline: source resolution capped at 1080p, SDR, HEVC Main + AAC stereo in MP4, with metadata at the front for streaming. Never upscale. Select quality and bitrate from real cartoon samples rather than inventing a universal setting. [S21, S25]
Encode strategy	Reuse compatible streams when possible. Otherwise normalize once in the background. FFmpeg provides hevc_videotoolbox; whether 1531Server can accelerate it must be checked against its hardware and installed build. Start with one low-priority job. [S22]
AV1 option	Swiftfin/VLCKit supports AV1, but client support is not proof of smooth decoding on this Apple TV. Compare source AV1 reuse against HEVC for startup, seek, frame drops and size. Avoid a costly AV1 re-encode until measurements justify it. [S20]
Validate	Probe streams, duration and dimensions; check decode integrity and audio/video sync; verify file digest and provenance. Mark our job complete only after validation. A download archive is a deduplication aid, not proof that publication succeeded. [S19]
Publish + play	Finalize MP4, NFO and approved artwork in staging. Move a complete bundle within the same RAID filesystem, then request indexing. Verify the actual Jellyfin playback session uses Direct Play rather than runtime transcoding.

OPTIMIZE FOR FAMILY PLAYBACK

Prefer the profile that yields reliable direct playback with acceptable quality and storage. Measure a representative episode on the real Apple TV when available; a simulator or a codec support table does not establish hardware performance.

No encode or device trial was run for this revision. Source formats, server CPU/encoder availability, client profile, audio and subtitles can change the Direct Play outcome. The first experiment should compare a small authorized sample, not ingest an entire channel.

One writable island on the RAID

The new cache exception does not relax protection of existing media.

SCOPE OF PERMISSION

Proposed root: /Volumes/<RAID>/KidsJellyFin-YouTube/ - an example, not a created path. A dedicated non-root worker may create and update only its own generated assets there. Every existing media path outside that root remains read-only. Do not change broad RAID ownership or permissions.

Location	Purpose and indexing boundary
Cache root / staging	Downloads, normalization and temporary bundles. Not a Jellyfin library root; keep it outside published Shows/Movies rather than relying on hidden-file behavior.
Cache root / published	Only complete approved series/episodes and their metadata. Jellyfin sees the explicit published/Shows folder, and published/Movies only if separately configured. Never index the whole cache or RAID root.
Cache root / quarantine	Unpublished service-owned assets awaiting parent review. Not indexed. Parent-controlled retention; no automatic purge of unknown or pre-existing files.
Server internal disk	Approval database, durable job queue, logs, credentials and configuration backups. None belong in the media library.

Survive interruption without harming the catalog

Persist jobs before work; resume or safely retry after a crash. Use source ID plus profile version for deduplication. A download failure affects that job, never the existing published episode. Do not overwrite the last good asset until its replacement is approved, validated and safely committed.

Verify the expected mounted volume identity and cache-root marker before any write. If the RAID is missing, wait and report unavailable; never create a replacement directory at its empty mount point. Reject symlinks and path escapes. Use OS permissions plus resolved-path checks, not string-prefix checks alone.

Bound storage and recovery work

Set a hard cache quota and free-space reserve, accounting for input, output and temporary overlap. Stop queuing when space is insufficient. Default to retained approved assets; any later automatic eviction must be opted into and restricted to manifest-owned cache files, never arbitrary files on the RAID.

Revocation requires an explicit unpublish operation, Jellyfin refresh and confirmation that the child can no longer access the item. Test active playback termination and stale item IDs; a file move alone does not prove immediate revocation. Back up the approval/job database and reconcile cache contents after reboot without broad cleanup.

The cache changes the comparison

Local transport makes the simplest child navigation much more attractive.

Preferred route: an approved playlist becomes local media

- Strawman.** Run yt-dlp against the playlist nightly, encode everything as AV1 and let Jellyfin scan the RAID. This confuses playlist trust with video approval, assumes a codec win and gives a worker unnecessary access.
- Steelman.** Freeze approved IDs, acquire permitted assets, normalize to a measured profile, publish verified bundles in one bounded writable folder, then use standard Jellyfin playback. Downloader updates and WAN outages affect new imports; already cached episodes remain stable.

Alongside the existing Shows shelf

- Strawman.** Call every matching video Little Bear and merge it into the existing series. Titles alone can mix clips, duplicate episodes and different sources.
- Steelman.** Publish one distinct, clearly labeled parent-curated series per playlist. Add its published root to Kid TV. Children use the familiar artwork, Next and Shuffle; provenance lives in metadata and parent controls. No extra player or source category is required.

A separate YouTube category

- Strawman.** Add a third Jellyfin library and expect the current kids app to discover it automatically. Its exact two-library access check deliberately rejects that configuration.
- Steelman.** Explicitly version the allowed-library binding and add an illustrated curated-source doorway. This distinguishes imported collections and provides source-specific admin status, at the cost of more navigation and client work. Keep it optional until child testing shows value.

Other route	Strongest case and practical limit
Official live player	Avoids a local mirror and uses supported YouTube playback. Its web-based player is not a ready tvOS integration, and recommendations remain a separate child-access issue. [S8, S9, S13]
Live resolver plugin	Closer to native playback, with less retained storage. URL expiry, WAN failures and extractor changes remain on the playback path. Cache ingestion moves those failures away from watching. [S3]
Licensed direct files	Same resilient local experience with clearer acquisition rights where a creator/distributor supplies permitted files. Availability of the exact desired shows remains unknown.

Recommendation: approved local-cache ingestion and the existing Shows shelf first. Revisit a third category as a product choice, not a technical prerequisite. Download permission remains distinct from the newly granted filesystem write permission. [S11, S17]

Make each playlist a stable local series

Use Jellyfin as the catalog and playback system; avoid a second player architecture.

Boundary	Implementation proposal
Parent import service	Own draft approval, source identities, jobs and publication history. Keep downloader/transcoder subprocesses behind a narrow worker boundary. Use validated IDs and fixed source hosts; no shell interpolation from playlist titles or arbitrary URL resolver.
Jellyfin layout	Represent an approved collection as a distinct series folder with tvshow.nfo, Season 01 and numbered files plus episode NFO. Supply approved local artwork and source provenance; lock curated fields to avoid incorrect external matching. [S23, S24]
Stable numbering	Assign positive import ordinals once. They represent collection order, not official broadcast seasons. Never renumber existing files because upstream playlist order changed. Version 1 preserves the approved initial order and appends newly approved videos.
Library access	Add only published/Shows as an extra folder of existing Kid TV. Do not change Kid Movie, enable all folders or add broad drive scans. A third curated library would require an explicit client binding migration and negative access tests. [S1]
Client responsibilities	Existing Jellyfin item IDs, catalog, player, Next, Shuffle and session limits remain the playback path. Keep source-to-local mapping in the import service. New child-facing code is limited to any deliberate display/provenance changes.
Progress and refresh	Reuse current Jellyfin user state plus client progress/shuffle persistence. Stable paths reduce identity churn; validate item-ID stability on scan and restart. Do not mark watched or overwrite progress through generated NFO. A cached item disappears from the app only after tested unpublication and refresh.

Adult metadata, child simplicity

A parent sees original title, channel, approval time, output codec, size and job errors. Children see approved collection artwork and the current Next/Shuffle controls. No import UI, YouTube search or Google authentication enters the kids flow.

THE FIRST VERSION CAN STAY SMALL

A parent web page, durable importer, protected cache folder and a precise Jellyfin library-folder addition are enough to trial the architecture. A new client source provider, YouTube player SDK and top-level category are unnecessary for this route.

No server metadata settings or account permissions were changed. Local NFO has priority over remote metadata in Jellyfin, but artwork and locked-field behavior must be verified for generated assets without changing settings for existing RAID media. [S23]

Test a small cache before scaling

All stages below are proposed implementation, not completed installation.

Stage	Required evidence
1. Readiness	Confirm exact RAID root, mounted volume identity, non-root write confinement, free-space budget and server encoder support. Establish permitted source material.
2. Small import	Approve a few videos; download, normalize and validate; publish one series with NFO/artwork. Confirm only the explicit cache subtree is indexed.
3. Playback	Compare HEVC and source AV1 where available. Measure startup, seeks, frame drops and Direct Play on the actual Apple TV when possible. Verify Next, Shuffle, resume and session limits.
4. Recovery + access	Exercise reboot, interrupted download/encode, WAN loss, absent RAID, low space, revoked approval, stale item IDs and a new upstream playlist member. Prove no writes outside the owned cache root.
5. Operations	Bound concurrency, retries and storage. Back up policy/jobs, provide parent-visible failures, and test tool upgrades with rollback. Review distribution obligations before release. [S14]

Success: a parent-approved collection plays through the ordinary Jellyfin path without live YouTube dependency; new playlist members cannot appear automatically; existing RAID media is unchanged; reboot recovery neither corrupts published files nor broadens library access.

Sources and verification

- [S1] Jellyfin: library administration

[S3] YouTubeSync: maintainer's implementation

[S5] YouTube Data API: playlistItems.list

[S6] YouTube Data API: video metadata

[S8] YouTube: IFrame Player API

[S9] YouTube: player parameters

[S11] YouTube: developer policies

[S13] Apple: WKWebView platform availability

[S14] Apple: App Review Guidelines
- [S17] YouTube: service terms

[S18] YouTube: editing without changing the URL

[S19] yt-dlp: formats, postprocessing and archive

[S20] Jellyfin: client codec compatibility

[S21] Apple TV 4K: supported video formats

[S22] FFmpeg: VideoToolbox HEVC encoder

[S23] Jellyfin: local NFO metadata

[S24] Jellyfin: TV series folder conventions

[S25] FFmpeg: MP4 and faststart options

Research reviewed 8 October 2026. Existing client access and playback profiles were inspected; no downloader, server encoder, cache or physical-device test ran. The earlier tvOS WebKit probe remains recorded in docs/design/youtube-tvos-feasibility.json, but this local-file proposal does not require a web player. All uncited behavior is proposed design.